

GPU Optimization of the “Isolated Ones” Stencil Problem

From CUDA Exploration to HIP Implementation

Konstantin Kolchin

June 7, 2026

Abstract

We study GPU kernels that count *isolated* cells marked 1 in a binary $N \times N$ matrix: a cell counts only if it equals 1 and all eight neighbors (including diagonals) are 0, with the matrix border treated as 0. Starting from a naïve per-thread implementation, we built a reproducible benchmark harness in a Jupyter notebook (Numba CUDA on Google Colab, NVIDIA T4), identified a tile-and-halo family of kernels (method 5) as the most promising direction, and ported it to a standalone C++/HIP application for AMD ROCm. The HIP port enabled LDS (shared-memory) micro-optimizations unavailable in the Python notebook stack—including explicit bank padding, `uint32` row packing, and warp shuffle—and was measured on an AMD Radeon RX 6800 XT (`gfx1030`). On NVIDIA, the naïve atomic baseline remains competitive for sparse inputs; on AMD, the tiled LDS approach wins clearly ($\sim 2.4\times$ at $N=1024$). Simple variants outperformed “textbook” bitwise and shuffle extensions on `gfx1030`, underscoring that optimizations must be validated per target architecture. Artifacts (notebook, HIP benchmark, plotting script) are at <https://github.com/konkolchin/cuda-to-hip> (private).

1 Problem statement

The problem about isolated ones was stated during the interview by Tomasz. Given a binary matrix G of size $N \times N$ (stored as `uint8`), count cells (i, j) such that

$$G_{i,j} = 1 \quad \text{and} \quad \forall (di, dj) \in \{-1, 0, 1\}^2 \setminus \{(0, 0)\} : G_{i+di, j+dj} = 0, \quad (1)$$

where out-of-bounds neighbors are defined as 0. This is **not** connected-component labeling: we count only cells that are “1” with an empty 3×3 neighborhood (excluding the center).

A literal GPU translation assigns one thread per interior cell and increments a global counter when the eight-neighbor test succeeds:

```
if (mat[i][j] == 1
    && mat[i-1][j-1] == 0 && mat[i-1][j] == 0 && mat[i-1][j+1] == 0
    && mat[i][j-1] == 0 && mat[i][j+1] == 0
    && mat[i+1][j-1] == 0 && mat[i+1][j] == 0 && mat[i+1][j+1] == 0)
    atomicAdd(&counter, 1);
```

This baseline is correct but inefficient on GPUs because of (i) branch divergence in the long `&&` chain, (ii) global `atomicAdd` serialization on hits, (iii) repeated uncached global loads for neighbor reads, and (iv) explicit boundary guards on every neighbor test.

Our goal was to explore alternative kernel structures, measure them under controlled sparsity regimes, and prepare a HIP implementation suitable for AMD hardware.

2 Phase I: CUDA exploration in Google Colab

2.1 Experimental setup

We implemented all kernels in Python using **Numba CUDA** inside a Jupyter notebook (`isolated_ones_gpu.co`) and executed benchmarks on **Google Colab** with an **NVIDIA Tesla T4** (compute capability 7.5, warp size 32).

Data layout. Matrices are always stored as dense `uint8` arrays of shape $N \times N$. For kernels that need a zero border without per-thread boundary branches, we also use a padded array of shape $(N+2) \times (N+2)$ with a one-cell zero halo.

Sparsity regimes. Each cell is independently set to 1 with probability equal to a *density* parameter. We benchmark three regimes:

- **sparse** (density = 0.01): few ones, many early exits;
- **medium** (density = 0.10);
- **dense** (density = 0.90): almost all cells are 1, few isolated.

Matrix sizes $N \in \{1024, 2048\}$; each kernel timing uses warmup iterations followed by repeated timed runs with device synchronization.

Correctness. Every GPU method is checked against a CPU reference implementation that implements the same eight-neighbor rule.

2.2 Method families explored

We catalogued several design points (names preserved in the benchmark output):

Table 1: Kernel families in the Colab notebook.

ID	Name	Idea
1	<code>interview_if_atomic</code>	Literal <code>if (&&...)</code> + global atomic
1b	<code>divergent_early_return</code>	Same logic, early <code>return</code> per check
2	<code>uniform_block_reduce</code>	Uniform neighbor-sum; per-block reduction
3	<code>compaction</code>	<code>flatnonzero</code> list of ones; sparse second pass
4	<code>warp_ballot</code>	Uniform path; full block reduce (ballot N/A in Numba)
5	<code>tile30_halo</code>	30×30 tile, 32×32 halo in shared memory
5b	<code>tile16_halo</code>	16×16 tile equals thread block (one thread/cell)
5c	<code>tile30_nbrcache</code>	Software cache: precompute 8-neighbor sum in LDS
5d	<code>tile30_block30</code>	30×30 tile equals 30×30 block (900 threads)
6	<code>lut_k3_warp</code>	$k=3$ bitmask LUT in shared memory (~32 MB table)

2.3 Why method 5 was selected

Among the “respectable fixes” (methods 2–5), **method 5** (`tile30_halo`) emerged as the most promising:

- It loads a **halo tile once** into shared memory (CUDA shared / AMD LDS) and reuses it for up to $30 \times 30 = 900$ interior cells per thread block.
- It avoids global atomics via a **block-local reduction**.
- It uses a **uniform neighbor-sum** test ($\sum \text{neighbors} = 0$) instead of a deeply divergent `&&` tree.
- At $N=1024$ it launched $\lceil N/30 \rceil^2 \approx 35^2 = 1225$ blocks—*fewer* than the $64^2 = 4096$ blocks of method 1—so it is not launch-bound in the same way as fine-grained bitmask/LUT schemes (method 6).

We applied a standard tiled-matrix lesson to reduce LDS bank conflicts: row stride `SM_STRIDE = HALO + 1 = 33` instead of 32 when indexing the 32×32 halo array.

2.4 Colab (NVIDIA T4) results

Figure 1 summarizes median kernel times from the notebook benchmark on the T4.

Key observations on T4. For **sparse** inputs at $N=1024$, the naïve atomic kernels (methods 1/1b) were often **fastest** or tied for first place: with only $\sim 1\%$ ones, most threads exit immediately and few atomics fire. Method 5 was typically within $\sim 10\text{--}20\%$ of method 1 (~ 0.46 ms vs. $\sim 0.38\text{--}0.55$ ms)—respectable, but not the overall winner on NVIDIA for sparse data. Method 6 (LUT) was substantially slower due to very large block counts and table pressure. These results motivated a deeper implementation on native GPU C++ where additional low-level features could be tested fairly.

3 Phase II: C++/HIP testing application

3.1 Motivation

The notebook was excellent for rapid prototyping, but Numba CUDA imposes limitations:

- No `__shfl_sync` / warp shuffle intrinsics in the Python kernels we wrote.
- No convenient `uint32` bitwise halo packing in device code.
- Coarse timing via host `time.perf_counter()` rather than `hipEvent` pairs on the device timeline.
- The CUDA *simulator* mode (when Colab GPU quota is exhausted) does not faithfully model atomics or device memory for full validation.

We therefore built `hip_isolated_ones`: a standalone C++17/HIP benchmark that reproduces the notebook’s correctness checks and timing methodology, targets **AMD ROCm** (tested on HIP 6.4), and adds kernel variants that exploit features available in native GPU C++.

3.2 Extensions in native C++/HIP (vs. the notebook)

Table 2 lists optimizations added in the HIP port but not exercised in the Colab notebook.

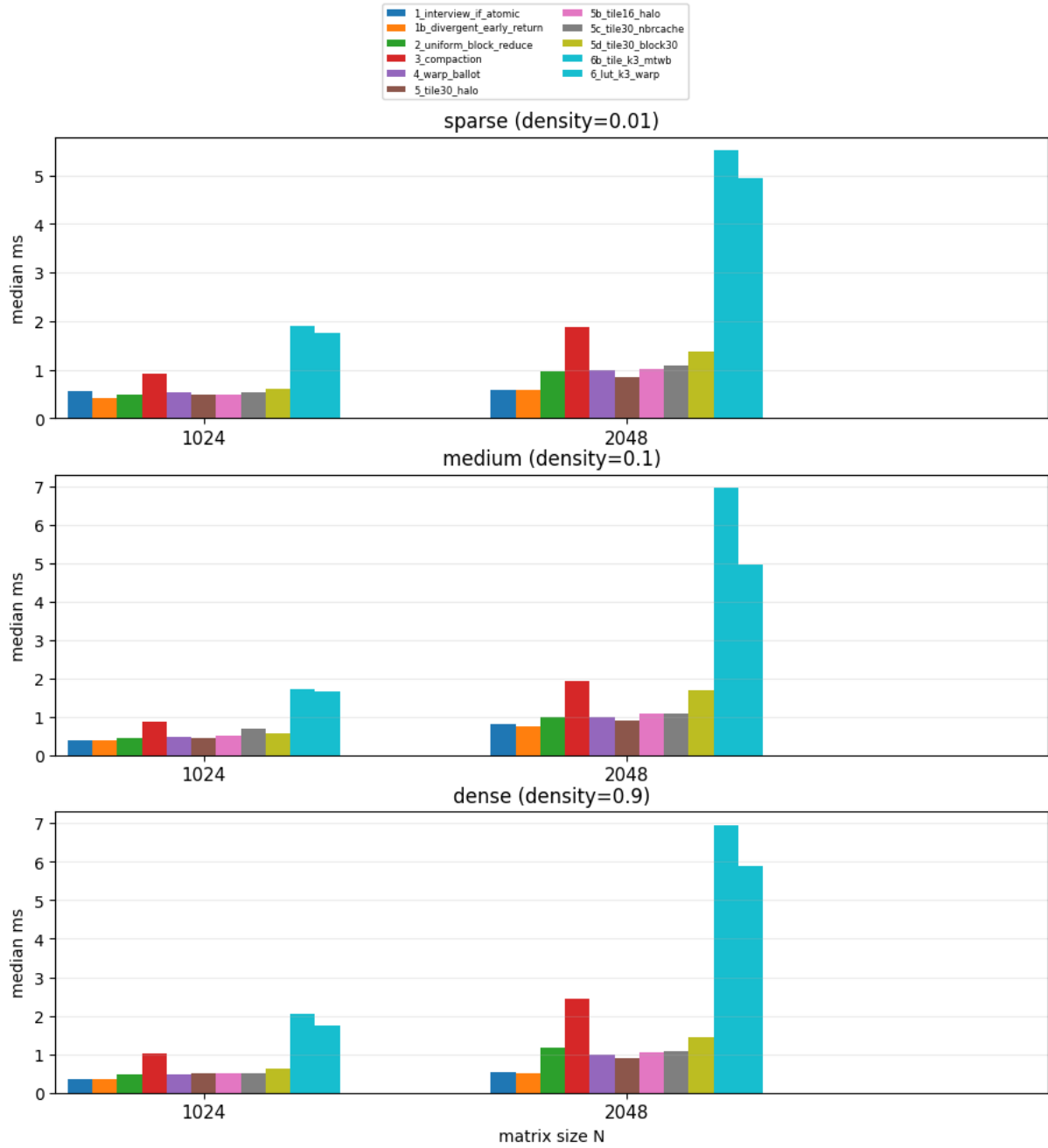


Figure 1: Median kernel time (ms) on **NVIDIA Tesla T4** (Google Colab), from the Jupyter/Numba CUDA notebook. One subplot per density regime; grouped bars for $N=1024$ and $N=2048$.

3.3 HIP-only features vs. CUDA portability

Short answer: we did not rely on any API that exists only in HIP and is absent from CUDA. HIP is deliberately a *CUDA-compatible* portability layer: the kernels use `__shared__`, `__syncthreads()`, `atomicAdd`, `__shfl`, and the triple-chevron launch syntax—all of which have direct or near-direct CUDA counterparts (`cudaEvent` vs. `hipEvent`, `__shfl_sync` vs. `__shfl`) [1, 2].

What the C++/HIP application added over the notebook was therefore:

- **Native device intrinsics** (`__shfl` with explicit width) not exposed in our Numba CUDA Python kernels [3];
- **Host runtime timing** via `hipEventRecord` / `hipEventElapsedTime` rather than host wall-clock timers [5];
- **Full control** over thread-block geometry (e.g. a (32, 8) block for variant 5g) and `uint32` bit tricks in device code.

AMD-specific context (terminology and hardware). On AMD GPUs, on-chip shared memory is called **LDS** (Local Data Share) [6]; our code still declares it with the portable `__shared__` keyword. HIP documents *cross-lane* operations (shuffle, ballot) for wavefronts; on many AMD parts the hardware wavefront is 64-wide, while our RX 6800 XT reported `warpSize=32` in device properties—shuffle width parameters (`width=16` or `32`) remain meaningful [3, 1]. These are portability and hardware-model considerations, not HIP-exclusive APIs that CUDA lacks. Further background is in the official ROCm/HIP documentation [4, 8, 7].

Table 2: Extensions in the HIP application beyond the notebook baseline.

Variant	Mechanism	Rationale
5a_tile30_nopad	Row stride = 32 (no +1 pad)	A/B test for LDS bank conflicts
5e_tile30_u32pack	Pack each 32-wide halo row into one <code>uint32</code>	Fewer, wider LDS accesses; bit tests for neighbors
5g_tile30_u32pack_shfl	<code>uint32</code> rows + <code>__shfl</code> for left/right	Pass horizontal neighbors between lanes
5f_tile16_shfl	Tile=block 16×16 + 16-wide shuffle groups	Clean mapping of row threads to shuffle width
(all)	<code>hipEvent</code> timing, 5 warmup runs	Stable GPU-side timing

3.4 Benchmark protocol on AMD

Hardware: **AMD Radeon RX 6800 XT**, architecture `gfx1030`, ROCm 6.4, `hipcc` (AMD clang 19). Reported `warpSize` from the runtime: **32** (relevant for shuffle width). Same matrix sizes, densities, and seeded random grids as Phase I. Each method: **5 untimed warmup** launches, then **20 timed** iterations; we record **median and mean** milliseconds (`hipEvent` elapsed time, including synchronization).

4 Results on AMD RX 6800 XT

Tables 3–8 list the median kernel times (ms) for all six benchmark scenarios on the RX 6800 XT: matrix sizes $N \in \{1024, 2048\}$ crossed with densities sparse (0.01), medium (0.10), and

dense (0.90). The same six scenarios are visualized in Figure 2; the bar chart omits 5e and 5g for readability (those rows appear in the tables).

4.1 Detailed median timings (six scenarios)

Table 3: RX 6800 XT: $N = 1024$, sparse (density = 0.01). Median kernel time (ms).

Method	median (ms)
1_interview_if_atomic	0.17
1b_divergent_early_return	0.17
2_uniform_block_reduce	0.17
5_tile30_halo	0.07
5a_tile30_nopad	0.07
5e_tile30_u32pack	0.12
5g_tile30_u32pack_shfl	0.10
5b_tile16_halo	0.07
5f_tile16_shfl	0.07
5c_tile30_nbrcache	0.07
5d_tile30_block30	0.08

Table 4: RX 6800 XT: $N = 1024$, medium (density = 0.10). Median kernel time (ms).

Method	median (ms)
1_interview_if_atomic	0.20
1b_divergent_early_return	0.21
2_uniform_block_reduce	0.07
5_tile30_halo	0.07
5a_tile30_nopad	0.07
5e_tile30_u32pack	0.11
5g_tile30_u32pack_shfl	0.10
5b_tile16_halo	0.07
5f_tile16_shfl	0.07
5c_tile30_nbrcache	0.07
5d_tile30_block30	0.09

4.2 Cross-platform comparison

5 Discussion

Method 5 wins on AMD. The tile-and-halo pattern amortizes one LDS load over hundreds of cells and replaces global atomics with block reductions. On `gfx1030` this dominates the naïve approach for *all* density regimes at $N=1024$, including sparse data where NVIDIA favored early-exit atomics.

Bank padding (5 vs. 5a). `SM_STRIDE = HALO + 1` is the standard remedy for 32-way shared-memory bank conflicts when warps walk along rows. On RX 6800 XT, 5 and 5a were **identical** within measurement resolution—the padding lesson is still worth stating in design reviews even when a particular kernel is not conflict-limited.

Bitwise pack and shuffle (5e, 5g, 5f). These extensions are natural “next steps” when moving from Python to HIP. On `gfx1030`, 5e/5g were **slower** than plain 5: the cost of building `uint32` row words and the altered thread-block geometry outweighed saved LDS traffic. 5f matched 5b—shuffle did not help for the 16×16 tile-equals-block layout at these problem sizes.

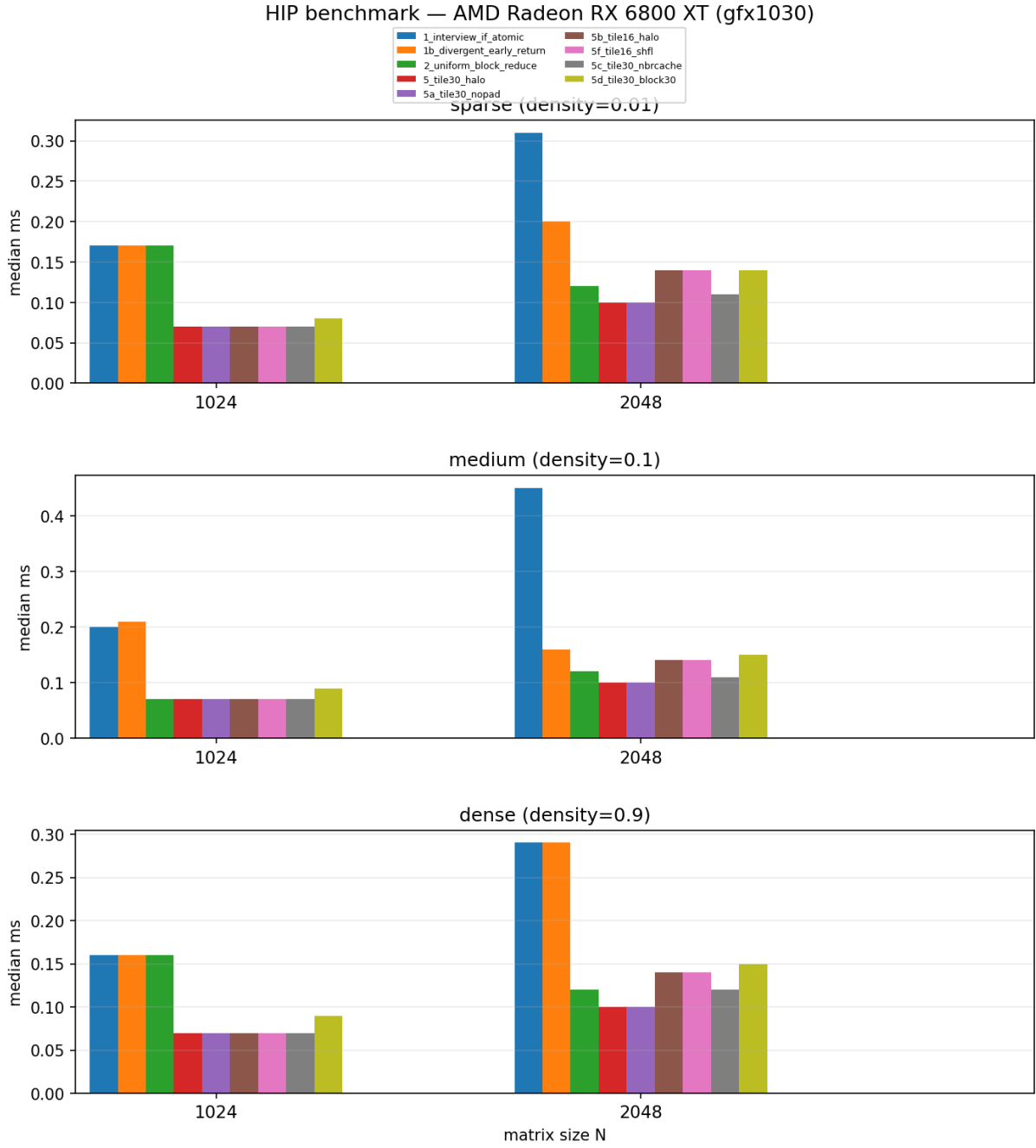


Figure 2: Median kernel time (ms) on **AMD Radeon RX 6800 XT** (gfx1030), from the C++/HIP application (`hip_isolated_ones`). One subplot per density regime; grouped bars for $N=1024$ and $N=2048$, corresponding to Tables 3–8. Variants 5e and 5g excluded from the plot (slower).

Table 5: RX 6800 XT: $N = 1024$, dense (density = 0.90). Median kernel time (ms).

Method	median (ms)
1_interview_if_atomic	0.16
1b_divergent_early_return	0.16
2_uniform_block_reduce	0.16
5_tile30_halo	0.07
5a_tile30_nopad	0.07
5e_tile30_u32pack	0.12
5g_tile30_u32pack_shfl	0.10
5b_tile16_halo	0.07
5f_tile16_shfl	0.07
5c_tile30_nbrcache	0.07
5d_tile30_block30	0.09

Table 6: RX 6800 XT: $N = 2048$, sparse (density = 0.01). Median kernel time (ms).

Method	median (ms)
1_interview_if_atomic	0.31
1b_divergent_early_return	0.20
2_uniform_block_reduce	0.12
5_tile30_halo	0.10
5a_tile30_nopad	0.10
5e_tile30_u32pack	0.22
5g_tile30_u32pack_shfl	0.21
5b_tile16_halo	0.14
5f_tile16_shfl	0.14
5c_tile30_nbrcache	0.11
5d_tile30_block30	0.14

Table 7: RX 6800 XT: $N = 2048$, medium (density = 0.10). Median kernel time (ms).

Method	median (ms)
1_interview_if_atomic	0.45
1b_divergent_early_return	0.16
2_uniform_block_reduce	0.12
5_tile30_halo	0.10
5a_tile30_nopad	0.10
5e_tile30_u32pack	0.23
5g_tile30_u32pack_shfl	0.21
5b_tile16_halo	0.14
5f_tile16_shfl	0.14
5c_tile30_nbrcache	0.11
5d_tile30_block30	0.15

Table 8: RX 6800 XT: $N = 2048$, dense (density = 0.90). Median kernel time (ms).

Method	median (ms)
1_interview_if_atomic	0.29
1b_divergent_early_return	0.29
2_uniform_block_reduce	0.12
5_tile30_halo	0.10
5a_tile30_nopad	0.10
5e_tile30_u32pack	0.23
5g_tile30_u32pack_shfl	0.22
5b_tile16_halo	0.14
5f_tile16_shfl	0.14
5c_tile30_nbrcache	0.12
5d_tile30_block30	0.15

Table 9: Qualitative ranking shift: sparse, $N=1024$.

Platform	Winner (sparse)	Method 5 vs. baseline
NVIDIA T4 (Colab)	Methods 1/1b $\approx$ best	~ 0.46 ms vs. ~ 0.4 ms (5 slightly slower)
AMD RX 6800 XT	Method 5 family	~ 0.07 ms vs. ~ 0.17 ms ($\sim \mathbf{2.4\times}$ faster)

Neighbor-sum cache (5c). Precomputing the eight-neighbor sum for every cell helps when many cells are 1 (dense), but always pays for empty cells. It tied with 5 at $N=1024$ and was marginally slower at $N=2048$.

Large thread blocks (5d). Using a $30\times 30 = 900$ -thread block removes the stride loop but increased sync pressure and slightly hurt performance vs. the 256-thread 5 kernel.

Method 1b anomaly at $N=2048$ medium. Early-return layout (1b) reported a much lower median than the single `if` form (1) in one full sweep (~ 0.16 ms vs. ~ 0.45 ms), while other regimes showed parity. With additional warmup and reporting both mean and median, 1b remained fast in that scenario; high variance in some repeated runs suggests sensitivity to GPU clock behavior for the atomic path under medium density. We treat **method 5** as the stable, architecture-agnostic headline result.

6 Conclusions

1. The isolated-ones stencil is a compact but rich GPU micro-benchmark: it exposes divergence, atomics, LDS tiling, occupancy, and cross-vendor ranking reversals.
2. Systematic exploration in a Colab Jupyter notebook identified **method 5 (tile30 halo + LDS + block reduce)** as the most promising structure, competitive on NVIDIA T4 but not always fastest on sparse inputs.
3. A native **C++/HIP** port on AMD RX 6800 XT validated that method 5 is **dominant on gfx1030** ($\sim 2.4\times$ over the atomic baseline at $N=1024$ sparse).
4. Native C++ device features (`uint32` packing, `__shfl`) must be profiled per GPU: on this hardware, simpler LDS tiling won. These intrinsics are available in CUDA as well; the gain was over the Python notebook, not over CUDA as a language.
5. Artifacts: Jupyter notebook (`isolated_ones_gpu_colab.ipynb`), HIP benchmark (`hip_isolated_ones/`), and plotting script (`plot_benchmark.py`) are version-controlled for reproducibility.

Artifacts and reproduction

- **Notebook:** `isolated_ones_gpu_colab.ipynb` — Numba CUDA kernels, CPU reference, Colab benchmark.
- **HIP app:** `hip_isolated_ones/` — build with `make ARCH=gfx1030`, run `./hip_isolated_ones`.
- **Plot:** `python plot_benchmark.py --input benchmark.txt` parses stdout and reproduces bar charts.
- **Repository:** <https://github.com/konkolchin/cuda-to-hip> (private; collaborators by invitation).

References

- [1] AMD. HIP frequently asked questions, 2020. Supported APIs; cross-lane shfl, ballot. URL: https://rocm.docs.amd.com/projects/HIP/en/docs-6.0.2/user_guide/faq.html.
- [2] AMD. CUDA device API supported by HIP, 2025. HIPIFY reference table. URL: https://rocm.docs.amd.com/projects/HIPIFY/en/latest/reference/tables/CUDA_Device_API_supported_by_HIP.html.
- [3] AMD. HIP C++ language extensions, 2025. Warp shuffle, shared memory, kernel launch. URL: https://rocm.docs.amd.com/projects/HIP/en/latest/how-to/hip_cpp_language_extensions.html.
- [4] AMD. HIP documentation, 2025. URL: <https://rocm.docs.amd.com/projects/HIP/en/latest/index.html>.
- [5] AMD. HIP runtime API — event management, 2025. hipEventRecord, hipEventElapsedTime. URL: https://rocm.docs.amd.com/projects/HIP/en/latest/reference/hip_runtime_api/modules/event_management.html.
- [6] AMD. Introduction to the HIP programming model, 2025. URL: https://rocm.docs.amd.com/projects/HIP/en/latest/understand/programming_model.html.
- [7] AMD. Porting CUDA code to HIP, 2025. URL: https://rocm.docs.amd.com/projects/HIP/en/latest/how-to/hip_porting_guide.html.
- [8] AMD. ROCm programming guide, 2025. URL: https://rocm.docs.amd.com/en/latest/how-to/programming_guide.html.